Project report · AI farming assistant for Tamil Nadu

VivasayiAI

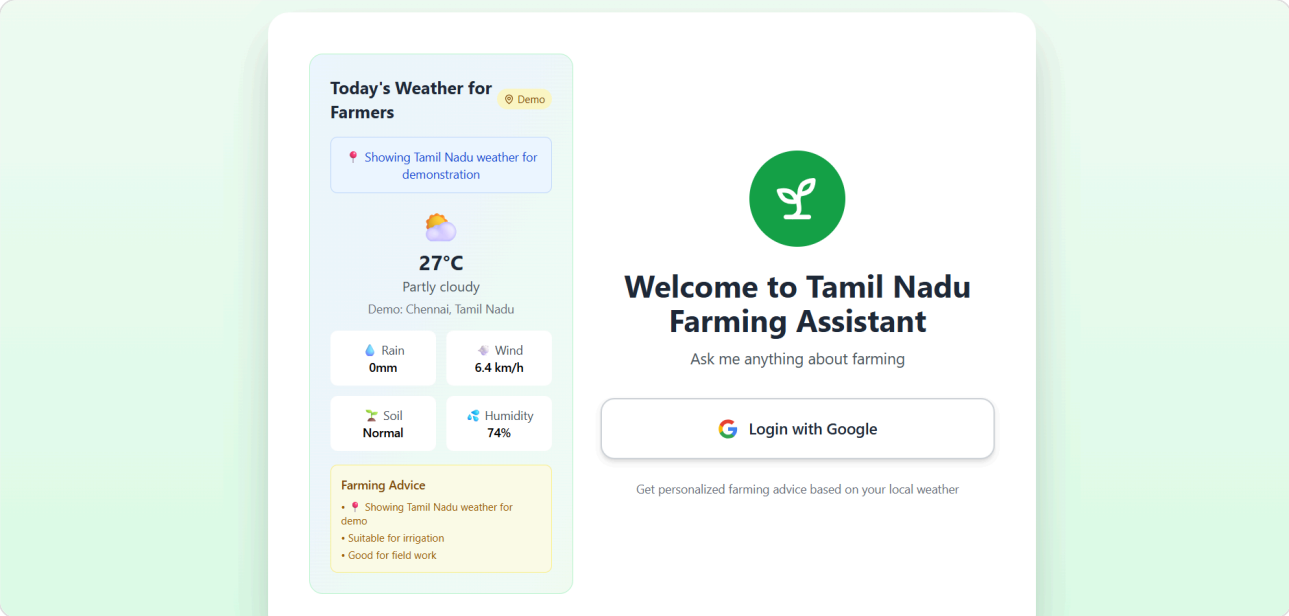
A bilingual (Tamil and English) assistant that gives farmers context-aware guidance, and helps them file and track crop-loss claims backed by checkable evidence.

Author
Kishore Balaji P

Date
November 2026

Backend
github.com/KishoreBalajiP/VivasayiAI

Frontend
github.com/KishoreBalajiP/VivasayiAI-Frontend



Contents

1. Executive Summary
2. Objectives and Scope
3. System Architecture
4. Tech Stack Overview
5. Features Overview
6. Folder Structure
7. Key API Endpoints
8. Deployment Overview
9. Implementation & Contributions
10. Challenges & Solutions
11. Future Enhancements
12. Conclusion
13. References

1 Executive Summary

VivasayiAI is an AI-powered farming assistant for farmers in Tamil Nadu. Farmers ask questions in Tamil or English and get answers grounded in curated agricultural data, their own farm profile, and live weather for their district.

Beyond advice, the platform helps a farmer file a crop-loss claim. The farmer marks the damaged area on a map and attaches photos and land documents. A background pipeline then gathers evidence from AI image analysis, document reading, weather history and satellite imagery. A deterministic rule engine turns that evidence into a decision. AI models produce evidence only; they never make the final decision.

2	5	1,135	0
languages, Tamil and English	evidence sources per claim	backend tests*	servers to manage (Lambda)

2 Objectives and Scope

Objectives

- **Localised guidance.** Advice based on the farmer's district, crops and current weather.
- **Native language.** A full experience in Tamil as well as English.
- **Grounded answers.** Retrieval-augmented generation over curated data, so the assistant does not invent facts.
- **Fair, checkable claims.** Crop-loss claims backed by independent evidence and an auditable decision.
- **Low running cost.** A serverless deployment that scales to zero when nobody is using it.

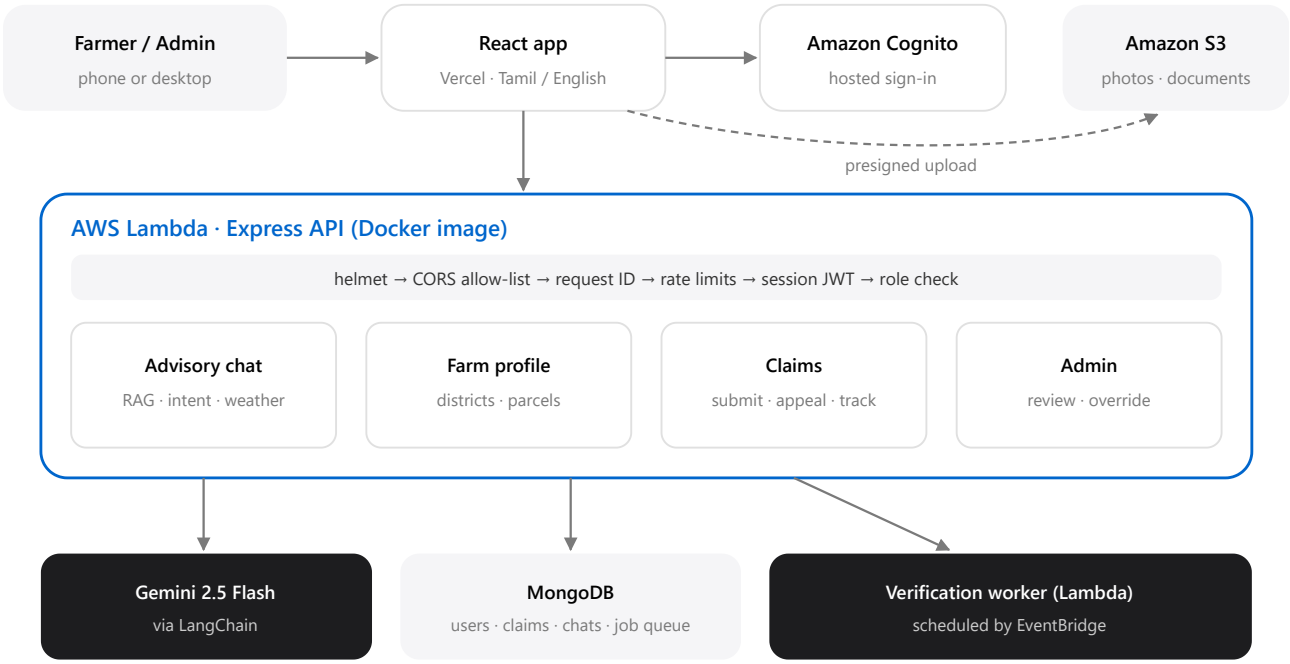
Scope

In scope: the advisory chat with image uploads, farm profiles and land parcels, the crop-loss claim lifecycle with evidence collection, an admin review queue, and farmer appeals.

The system supports agricultural officers and does not replace them. Claim outcomes have been validated only against test data so far, and they are not yet used for real compensation decisions.

3 System Architecture

The front end is a React app on Vercel. The backend is an Express app packaged as a Docker image and run on **AWS Lambda**. A second, scheduled Lambda processes verification jobs. Users sign in through **Amazon Cognito**, files go straight to **Amazon S3** through presigned URLs, and MongoDB stores all application data.

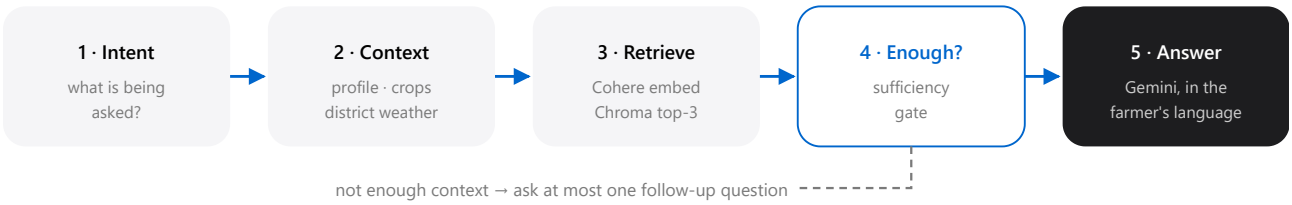


Also used: Chroma Cloud (vectors) · Cohere embeddings · Open-Meteo (forecast and archive weather) · Copernicus Sentinel-2 (satellite imagery).

Figure 1. System architecture.

Advisory chat pipeline

Every message goes through a fixed sequence of steps. The assistant asks one follow-up question when it lacks enough context, and retrieved text is fenced as untrusted data so it cannot steer the model.



If the AI provider is over its quota, the user gets a clear "service busy" message instead of a timeout.

Figure 2. Advisory chat pipeline.

Crop-loss claim verification

Submitting a claim queues a verification job. The worker collects evidence step by step, retrying failed steps with increasing back-off. A versioned rule engine then makes the decision and records which policy produced it.

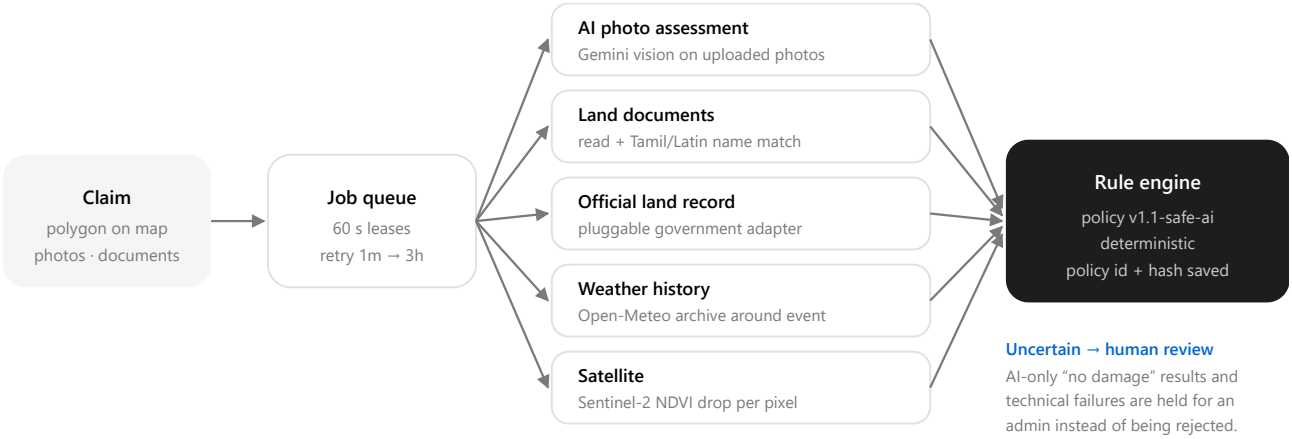


Figure 3. Claim verification. Evidence sources inform the decision but cannot reject a claim on their own.

4 Tech Stack Overview

Frontend

- React and TypeScript
- Vite
- Tailwind CSS
- Tamil and English translations
- Playwright end-to-end tests

Backend

- Node.js 24 and Express
- MongoDB with Mongoose
- Zod validation and Pino logging
- Sharp for image processing
- Vitest with mongodb-memory-server

AI & data

- Google Gemini 2.5 Flash through LangChain
- Cohere `embed-english-v3.0` embeddings
- Chroma Cloud vector store
- Open-Meteo forecast and archive APIs
- Copernicus Sentinel-2 imagery

Cloud & DevOps

- AWS Lambda (container image) and Amazon ECR
- Amazon Cognito and Amazon S3
- Amazon EventBridge with CloudFormation
- GitHub Actions CI/CD
- Vercel for the front end

5 Features Overview

5.1 Farmer-facing features

- **Sign-in.** Secure sign-in through Amazon Cognito's hosted page.
- **Language choice.** The whole app and the AI's replies are available in Tamil or English.
- **AI chat.** Questions about crops, weather, irrigation and practices, with saved chat history.
- **Photo diagnosis.** Farmers can attach a crop photo for the AI to look at.
- **Weather-aware advice.** Forecasts for the farmer's district are included automatically.
- **Farm profile.** District, crops, and land parcels drawn on a map.
- **Crop-loss claims.** A 5-step wizard to file a claim, plus submit, withdraw and resubmit.
- **Progress tracker and appeals.** Farmers can see where a claim stands and appeal a decision.

5.2 Admin and system features

- A review queue and dashboard for claims that need a human decision
- Overrides with a reason, and a re-queue option for failed verification jobs
- Ownership and overlap checks that flag claims for review rather than accusing anyone of fraud
- A full audit trail: every decision stores its policy id, policy hash and evidence references
- Per-user rate limits, a request size limit and a CORS allow-list

6 Folder Structure

```
backend/
  app.js, index.js   Express app + Lambda handler
  routes/            auth, chat, claims, admin…
  controllers/       request handling
  middlewares/       auth, roles, rate limits
  services/          claims, verification engine
    satellite/       Sentinel-2 processing
  src/ai/            intent, context, prompts
  rag/               chunking + ingestion
  models/            Mongoose schemas
  infra/             CloudFormation (worker)
  tests/             Vitest suites
  Dockerfile
```

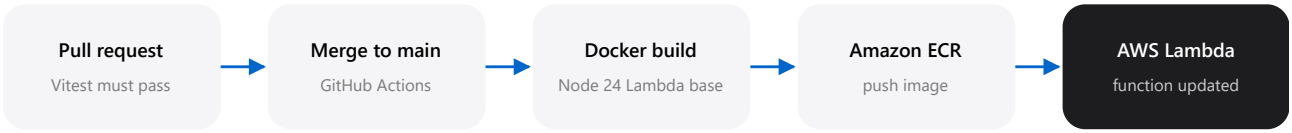
```
frontend/src/
  components/        chat, claims wizard, map
  context/           auth + session state
  services/          API + weather clients
  config/            district coordinates
  i18n.ts            Tamil / English strings
  types.ts           shared interfaces

.github/workflows/
  deploy-lambda.yml  test → build → ECR → Lambda
```

7 Key API Endpoints

Area	Endpoint	Purpose
Auth	`POST /auth/google`	Exchange the Cognito code for a session
Chat	`POST /chat/new`	Start a conversation
	`POST /chat/:id/message`	Send a message, optionally with an image
	`GET /chatsessions`	List or delete saved sessions
Weather	`GET /weather`	District forecast, cached
Profile	`GET /profile`	Read the farm profile
	`POST /profile/parcels`	Register a land parcel polygon
Uploads	`POST /upload/presign`	Get a presigned S3 URL
Claims	`POST /claims`	Create a draft claim
	`POST /claims/:id/submit`	Submit the claim and queue verification
	`POST /claims/:id/withdraw` , `/resubmit`	Manage the claim lifecycle
	`GET /claims/:id/progress`	Status for the farmer's tracker
	`POST /claims/:id/appeal`	Appeal a decision
Admin	`GET /admin/dashboard`	Review queue overview
	`POST /admin/claims/:id/override`	Make a human decision with a reason
	`POST /admin/claims/:id/jobs/requeue`	Retry a failed verification job

8 Deployment Overview



The verification worker and its EventBridge schedule are defined in CloudFormation (infra/verification-worker.yaml).

Figure 4. Backend CI/CD pipeline.

- **Frontend:** Vercel
- **Backend:** AWS Lambda (container image) behind an HTTPS endpoint
- **Background jobs:** a worker Lambda triggered by an EventBridge schedule
- **Auth and storage:** Amazon Cognito and Amazon S3
- **Data:** MongoDB and Chroma Cloud
- **CI/CD:** GitHub Actions, from tests through to ECR and Lambda

9 Implementation & Contributions

As the sole developer of VivasayiAI, I:

- Designed the serverless architecture and the claim verification workflow
- Built the React front end with full Tamil and English support
- Built the Express API, the MongoDB data model and the access controls
- Built the RAG pipeline: data chunking, embeddings, retrieval and grounded prompts
- Implemented the verification job queue, the five evidence steps and the versioned rule engine
- Integrated Cognito, S3 presigned uploads, Open-Meteo and Sentinel-2 imagery
- Containerised the backend and automated deployment with GitHub Actions, ECR and Lambda
- Wrote 1,135 backend tests and 90 Playwright end-to-end tests

1,135	90	10	6
backend tests*	end-to-end tests*	claim states, frozen	retry levels per job

* As recorded in the repository README in October 2026.

10 Challenges & Solutions

Challenge	Solution
AI accuracy	Answers are grounded with retrieval, a sufficiency gate asks a follow-up instead of guessing, and retrieved text is fenced as untrusted data.
AI deciding claims	Models only produce evidence. A deterministic, versioned rule engine decides, and uncertain cases go to a person.
Tamil text	Inputs and outputs are handled in Unicode, and documents are matched on both Tamil and Latin spellings of names.
Unreliable providers	Job leases, retries with back-off from 1 minute to 3 hours, dead-job review cases, and a clear “busy” response when the AI quota runs out.
Large uploads on Lambda	Files go straight from the browser to S3 through presigned URLs and never pass through the function.
Missing data sources	There is no public Tamil Nadu land-record API, so the adapter reports “unavailable” rather than inventing a match.

11 Future Enhancements

- Voice input and spoken answers in Tamil
- Market price integration
- SMS and WhatsApp channels for farmers without smartphones
- Expert-reviewed advisory content in the knowledge base
- Calibrating the satellite thresholds against real field data

- An official land-record integration once an authorised API is available

12 Conclusion

VivasayiAI combines local-language AI advice with a careful, auditable claims process. Its serverless design keeps running costs close to zero when idle. Above all, it treats AI as a source of evidence and leaves decisions to explicit rules and human reviewers, which matters when the outcome affects a farmer's income.

13 References

1. LangChain documentation — docs.langchain.com
2. AWS — What is Retrieval-Augmented Generation — aws.amazon.com
3. AWS Lambda container images — docs.aws.amazon.com/lambda
4. Amazon Cognito — docs.aws.amazon.com/cognito
5. Chroma — trychroma.com
6. Open-Meteo API — open-meteo.com/en/docs
7. Copernicus Data Space (Sentinel-2) — dataspace.copernicus.eu
8. Google Gemini API — ai.google.dev